

Expressiveness of notations for concurrency: reconsidering some prejudices!

Cliff Jones

Newcastle University

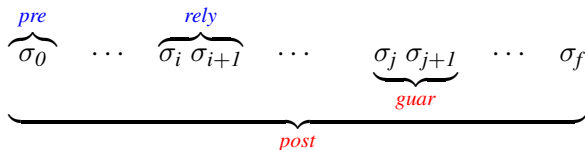
Ch Meyer (@Villebrumier)
2016-10-06

Background

- Rely/Guarantee (R/G)
 - [Jon81, Jon83a, Jon83b] used keywords (VDM style)
 - could be presented as $\{P, R\} S \{G, Q\}$
 - useful intro [Jon96]
 - many uses — including [BA10, BA11]
 - with Hayes: refinement calculus style [HJC14, JHC15]
 - Ian's newer semantics gets more out of algebraic properties
- simple ideas — but requires adopting an R/G mindset!

Rely/Guarantee (R/G) idea is simple

face interference (in specifications and design process)



- assumptions *pre/rely*
- commitments *guar/post*
- idea is rather general — cf. HJJ

rely relations are an abstraction of interference to be tolerated

My prejudices

- post conditions should be relations [Jon80, Jon86]
 - well-founded loop relation vs. “variant function” arguments
- data abstraction/reification is at least as important as control structures
- this is reinforced in concurrency [Jon07, JY15]
- compositionality is mandatory
- “ghost” variables dent/destroy compositionality
 - in the extreme, they can dictate the entire environment code
 - in many cases, they can be avoided
 - if I have to use them, I want(ed) a test
(cf. homomorphic rule vs. “biased” specifications)

Expressivity

- R/G is expressively weak
 - only relations
 - reflexive (for no interference); transitive (for multiple steps)
- maybe the weakness is their strength?
 - simple POs
- notation for “possible values” ($x \in \hat{y}$) [JP11, JH16]

(Concurrent) garbage collection

Abstract specification

$\Sigma_0 :: \text{busy: Addr-}\mathbf{set}$
 $\text{free: Addr-}\mathbf{set}$

where

$\text{inv-}\Sigma_0(\text{mk-}\Sigma_0(\text{busy}, \text{free})) \triangleq \text{busy} \cap \text{free} = \{ \}$

$\text{garbage}(\text{busy}, \text{free}) \triangleq \text{Addr} - (\text{busy} \cup \text{free})$

Collector

wr *free*

rd *busy*

pre **true**

$\vdots$

post $\text{garbage}(\text{busy}, \text{free}) \subseteq \widehat{\bigcup \text{free}}$

Two data reifications get us to:

disable interrupts

$\Sigma_I \text{ :: } roots: Addr\text{-}\mathbf{set}$
 $hp: Heap$
 $free: Addr\text{-}\mathbf{set}$

where

$inv\text{-}\Sigma_I(mk\text{-}\Sigma_I(roots, hp, free)) \triangleq$
 $\mathbf{dom} \ hp = Addr \wedge$
 $free \cap reach(roots, hp) = \{ \}$

upper bound for GC

$Heap = Addr \xrightarrow{m} Node$

$Node = Addr^*$

Simplification: here **nil** is ignored

Marking

(Sequential) garbage collection

Collector $\triangleq$ (*Unmark-s*; *Mark-s*; *Sweep-s*)

Mark-s

ext wr *marked*

rd *hp, roots, free*

pre true

post *marked'* = *reach*(*roots*, *hp'*)

Non-interfering case

$Mark-s \triangleleft marked \leftarrow roots$; **while** new marking **do** $Body$ **od**

$Body \triangleleft$

local $consid: Addr$ -**set**;

$consid \leftarrow \{ \}$;

while $consid \neq Addr$ **do**

let $x \in (Addr - consid)$ **in**

if $x \in marked$ **then** $Mark-kids(x)$ **else skip**;

$consid \leftarrow consid \cup \{x\}$

od

The loop proof (uses relation) and *to-end* argument

(Unrealistic) atomicity

(Concurrent) garbage collection

$$< hp(a), marked \leftarrow hp(a) \upharpoonright \{i \mapsto b\}, marked \cup \{b\} >$$

Previous loop proof goes through almost unchanged

enable interrupts

Concurrent case

I have worked on “fiction of atomicity/atomicity refinement”
but I don’t think that’s the right approach here

The *Mutator* cannot be atomic

$$hp(a) \leftarrow hp(a) \dagger \{i \mapsto b\}; marked \leftarrow marked \cup \{b\}$$

the essence of the problem is a “three state” argument:

Mutator can change $hp(a)$ (at i) to point to b

but then go to sleep

if the *Collector/Body* has already passed a , its (obvious) post condition fails

but we will get to another pointer to b

unless the *Mutator* destroys that link

but in that case, the *Mutator* must have marked b to get to that point

Of course, we could just reject the restrictions of R/G and use
(some) Temporal Logic

Concurrent case

But, what if I concede a ghost variable?

$Tbm :: p:Addr$	parent
$k:Addr$	kid

$\langle hp(a), tbm \leftarrow hp(a) \uparrow \{i \mapsto b\}, mk-Tbm(a, b) \rangle;$
 $\langle marked, tbm \leftarrow marked \cup \{b\}, \mathbf{nil} \rangle$

Concurrent case

Body

wr *marked*

rd *hp*

pre *true*

rely

$marked \subseteq marked' \wedge$

$(\forall a \in Addr, i \in \mathbf{inds} \ hp(a) \cdot$

$hp'(c)(i) \neq hp(c)(i) \Rightarrow hp'(a)(i) \in reach(roots, hp)) \wedge$

guar-Mutator

guar

$free \subseteq free' \wedge \dots$

post

$marked \cup \bigcup \{ \mathbf{elems} \ hp'(a) \mid a \in marked \} \subseteq$

$marked' \cup \{ tbm'.k \mid tbm' \neq nil \}$

Concurrent case

guar-Mutator(*Redirect*(*a*, *i*, *b*)):

$$tbm = \mathbf{nil} \wedge tbm' = mk\text{-}Tbm(a, b) \wedge marked' = marked \wedge$$

$$hp' = hp \upharpoonright \{a \mapsto hp(a) \upharpoonright \{i \mapsto b\}\} \vee$$

$$tbm \neq \mathbf{nil} \wedge tbm' = \mathbf{nil} \wedge marked' = marked \cup \{tbm.k\} \wedge hp' = hp$$

enables(?) proof of *post-Body*

Holy cow #1 is sacrificed (I conceded a ghost variable)
 but I do have a test

Transitive R/G?

Furthermore *guar-Mutator* is not transitive!

but it lets me argue about **two** half cases
(and their transitive closure)

Should/need you care about my changing prejudices?
Up to you — but I like my Mountauban proofs
Comments welcome



Richard Bornat and Hasan Amjad.

Inter-process buffers in separation logic with rely-guarantee.

Formal Aspects of Computing, 22(6):735–772, 2010.



Richard Bornat and Hasan Amjad.

Explanation of two non-blocking shared-variable communication algorithms.

Formal Aspects of Computing, pages 1–39, 2011.



Ian J. Hayes, Cliff B. Jones, and Robert J. Colvin.

Laws and semantics for rely-guarantee refinement.

Technical Report CS-TR-1425, Newcastle University, July 2014.



Cliff B. Jones and Ian J. Hayes.

Possible values: Exploring a concept for concurrency.

Journal of Logical and Algebraic Methods in Programming, 2016.



Cliff B. Jones, Ian J. Hayes, and Robert J. Colvin.

Balancing expressiveness in formal approaches to concurrency.

Formal Aspects of Computing, 27(3):465–497, 2015.



C. B. Jones.

Software Development: A Rigorous Approach.

Prentice Hall International, Englewood Cliffs, N.J., USA, 1980.



C. B. Jones.

Development Methods for Computer Programs including a Notion of Interference.

PhD thesis, Oxford University, June 1981.

Printed as: Programming Research Group, Technical Monograph 25.



C. B. Jones.

Specification and design of (parallel) programs.

In *Proceedings of IFIP'83*, pages 321–332. North-Holland, 1983.



C. B. Jones.

Tentative steps toward a development method for interfering programs.
Transactions on Programming Languages and System, 5(4):596–619, 1983.



C. B. Jones.

Systematic Software Development Using VDM.
Prentice Hall International, 1986.



C. B. Jones.

Accommodating interference in the formal design of concurrent object-based programs.
Formal Methods in System Design, 8(2):105–122, March 1996.



C. B. Jones.

Splitting atoms safely.
Theoretical Computer Science, 375(1–3):109–119, 2007.



Cliff B. Jones and Ken G. Pierce.

Elucidating concurrent algorithms via layers of abstraction and reification.
Formal Aspects of Computing, 23(3):289–306, 2011.



Cliff B. Jones and Nisansala Yatapanage.

Reasoning about separation using abstraction and reification.
In Radu Calinescu and Bernhard Rumpe, editors, *Software Engineering and Formal Methods*, volume 9276 of *LNCS*, pages 3–19. Springer, 2015.